PART 1

React Interview Questions & Answers

Complete Guide for Technical Interviews

Table of Contents

1. Core Concepts & Fundamentals

1. What is the key difference between the Real DOM and the Virtual DOM?

The **Real DOM** is a structured representation of the UI in the browser's memory. Updating it is slow because it causes a reflow and repaint of the page.

The **Virtual DOM** is a lightweight, in-memory JavaScript object representation of the Real DOM. When state changes, React creates a new Virtual DOM tree, compares it with the previous one using a "diffing" algorithm, and then efficiently updates only the changed nodes in the Real DOM. This process, called reconciliation, makes updates much faster.

2. Explain the reconciliation process in React. How does React decide to re-render?

Reconciliation is React's algorithm for diffing one Virtual DOM tree with another to determine which parts need to be updated.

- **Trigger:** A render is triggered by a state or prop change.
- **Virtual DOM Creation:** React creates a new Virtual DOM tree for the component.
- **Diffing:** React compares this new tree with the previous one.
- **Heuristics:** It uses efficient heuristics, assuming that different types of elements will produce different trees, and that elements of the same type can be updated. Keys are used to optimize list diffing.
- **Commit:** Finally, React applies the minimal set of changes to the Real DOM.

3. What is JSX? How is it transformed into JavaScript that the browser can understand?

JSX is a syntax extension for JavaScript that looks like HTML. It provides a more readable way to describe the UI structure. JSX is not valid JavaScript. Tools like Babel transpile JSX into `React.createElement()` function calls, which return plain JavaScript objects (React elements) describing what should be rendered.

4. Why can't browsers read JSX directly?

Browsers only understand standard JavaScript. JSX is a syntactic sugar that needs to be compiled down to `React.createElement()` calls, which are valid JavaScript that create objects describing the UI.

5. What are the rules of JSX?

- **Single Parent Element:** You must return a single parent element (e.g., a , <>, or <React.Fragment>).
- **className not class:** Use className instead of class to avoid conflicts with the JavaScript class keyword.
- **htmlFor not for:** Use htmlFor instead of for in labels.

- **CamelCase Properties:** Use camelCase for most attributes (e.g., onClick, tabIndex).
- **Self-Closing Tags:** Tags with no children must be self-closed (e.g., <img />).
- **JavaScript Expressions:** Embed JavaScript expressions using curly braces {}.

6. What is the significance of keys in React lists? What happens if you use an index as a key and the list order changes?

Keys help React identify which items have changed, are added, or are removed. They should be stable, unique, and predictable. Using the array index as a key is an anti-pattern when the list can be reordered, filtered, or items can be added/removed from anywhere but the end. If the list order changes, the index for each item changes. React will re-render and potentially re-create DOM elements for all items, leading to performance issues and state bugs (e.g., input field state getting mixed up).

7. What is the difference between an Element and a Component in React?

A **React Element** is a plain object describing what you want to see on the screen. It is immutable and cheap to create. `React.createElement('div')` returns an element.

A **React Component** is a function or class that optionally accepts inputs (props) and returns a React element tree.

8. Can you call a React hook inside a loop or a conditional? Why or why not?

No. Hooks must be called at the top level of your React function, before any early returns. This rule ensures that Hooks are called in the exact same order each time a component renders. React relies on this order to preserve the state of Hooks between multiple useState and useEffect calls.

9. What is the purpose of React.Fragment?

`React.Fragment` (<>...</>) lets you group a list of children without adding extra nodes to the DOM. It's useful to satisfy the "single parent element" rule of JSX without

introducing an unnecessary wrapper div.

10. What is the second argument you can pass to React.createElement?

The second argument is an object containing the props (including children). For example: `React.createElement('h1', { className: 'title' }, 'Hello World')`.

11. How does React handle user-defined component names that start with a lowercase letter?

React treats components starting with lowercase letters as built-in DOM components (like ,). If you define a component starting with a lowercase letter (e.g., myComponent), and use it as <myComponent />, React will look for a DOM tag named mycomponent and fail. Always start component names with an uppercase letter.

12. What is a "Pure Component" in the context of React?

`React.PureComponent` is a class component that implements a shallow comparison of props and state in its shouldComponentUpdate method. If no changes are detected, it prevents a re-render, boosting performance. For function components, the equivalent is `React.memo`.

13. What is the strict meaning of "children" in React?

children is a special prop that is passed to a component, containing the content between its opening and closing tags. It can be a string, a JSX element, an array of elements, or even a function (as in the "render props" pattern).

14. What is the significance of the defaultProps property?

defaultProps is a property on a class component (or a function component) that allows you to set default values for props in case they are not provided by the parent component. With modern JavaScript, default parameters are often used instead.

15. Explain the concept of "Lifting State Up" in React.

When several components need to share the same changing data, you lift the shared state up to their closest common ancestor. This single source of truth is then passed down to the children via props. This is a fundamental pattern for sharing state between siblings.

2. Components, Props, and State

16. What is the difference between state and props?

- **Props:** Are read-only, immutable inputs to a component passed down from a parent. They allow a component to be configurable.
- **State:** Is mutable data that is local and fully controlled by the component itself. State changes over time, usually in response to user events, and triggers a re-render.

17. Why should you not update the state directly? What should you do instead?

Directly mutating state (e.g., `this.state.count = 5`) does not notify React that a re-render is needed. You must use the setter function (setState for classes, the state setter from useState for functions). This ensures the component re-renders and the UI stays in sync with the data.

18. How does setState behave? Is it synchronous or asynchronous?

setState is asynchronous. React may batch multiple setState calls for performance reasons. This means you cannot rely on the updated state value immediately after calling setState. To perform an action after the state has been updated, use the callback function provided as the second argument to setState.

19. What will happen if you use props in the initial state of a component?

Using props to set the initial state (e.g., `state = { color: props.initialColor }`) is an anti-pattern. The state will only be initialized with the prop value when the component is first created. If the parent component re-renders and passes a new initialColor prop, the child's state will not update. The state becomes a "fork" of the original prop. The correct solution is to either make the component controlled (fully by props) or use the key prop to reset internal state.

20. What are "Controlled" vs "Uncontrolled" components?

- **Controlled Component:** Form data is handled by the React component state. The input's value is tied to state, and changes are handled via event handlers (like onChange). The React state is the "single source of truth."
- **Uncontrolled Component:** Form data is handled by the DOM itself. You use a ref to access the input value directly from the DOM. The DOM is the source of truth.

21. How can you pass data from a child component to a parent component?

By passing a function from the parent to the child as a prop. The child can then call this function, passing the data as an argument.

22. What is the purpose of the prop-types library?

prop-types is a library for type-checking the props passed to a component. It allows you to define the expected type and shape of each prop, and React will warn you in development if the wrong type is provided.

23. What is the difference between a functional component and a class component?

- **Functional Component:** A plain JavaScript function that accepts props and returns JSX. Before Hooks, they were stateless. Now, with Hooks, they can use state and lifecycle features.
- **Class Component:** An ES6 class that extends React.Component. It has a render method and can hold and manage local state and have lifecycle methods.

24. Can you force a component to re-render without calling setState?

In class components, you can call `this.forceUpdate()`, but this is highly discouraged and should be avoided as it breaks React's predictable data flow. In function components,

you can use a hack like the useState updater function with a dummy state: `const [_, forceUpdate] = useState({}); forceUpdate({});`.

25. What will happen if you use setState in the constructor?

The component hasn't been mounted yet, and its state is being initialized. Calling setState here is unnecessary and can lead to errors or unexpected behavior. You should assign the initial state directly to `this.state` in the constructor.

26. How can you prevent a component from rendering?

By returning null from its render method (or the functional component itself). You can conditionally return null based on the component's state or props.

27. What is the correct way to update an object in state?

You must create a new object or a copy of the existing object. Never mutate the state object directly.

For Classes: `this.setState(prevState => ({ user: { ...prevState.user, name: 'New Name' } }))`

For Functions: `setUser(prevUser => ({ ...prevUser, name: 'New Name' }))`

28. What is the correct way to update an array in state?

Similarly, you must create a new array. Do not use methods that mutate the original array like push or pop.

- **Add:** `setItems(prevItems => [...prevItems, newItem])`
- **Remove:** `setItems(prevItems => prevItems.filter(item => item.id !== idToRemove))`
- **Update:** `setItems(prevItems => prevItems.map(item => item.id === idToUpdate ? { ...item, ...updatedData } : item))`

29. Explain the concept of "Prop Drilling" and its potential issues.

Prop drilling is the process of passing props down through multiple layers of components to get them to a deeply nested component that needs them. The issue is that intermediary components are forced to accept and pass along props they don't use themselves, making the code harder to maintain and refactor. Solutions include Context API or state management libraries.

30. What is the difference between creating a component with React.createClass and an ES6 class?

React.createClass was the old way to create classes and is now deprecated. ES6 classes are the modern standard. Key differences were auto-binding of methods in createClass (requiring manual binding in ES6 classes) and a different syntax for initial state and propTypes.

3. The Component Lifecycle & useEffect

31. What are the main phases of the React component lifecycle?

- **Mounting:** Component is being created and inserted into the DOM.
- **Updating:** Component is re-rendering due to changes in props or state.
- **Unmounting:** Component is being removed from the DOM.

32. What are the most commonly used lifecycle methods in a class component?

- **componentDidMount:** Called after the component is mounted. Ideal for API calls, subscriptions, or DOM manipulations.
- **componentDidUpdate:** Called after the component's updates are flushed to the DOM. Good for DOM operations after a prop/state change.
- **componentWillUnmount:** Called just before the component is unmounted. Used for cleanup (e.g., invalidating timers, canceling network requests, removing subscriptions).

33. Why is componentDidMount a good place to make API calls?

It runs after the component has been rendered to the DOM for the first time. This ensures that the component is present before you try to update its state with the fetched data, which would trigger a re-render.

34. What is the equivalent of componentDidMount in a functional component?

The useEffect hook with an empty dependency array: `useEffect(() => { /* code here */ }, [])`.

35. How can you mimic componentDidUpdate using useEffect?

By using useEffect without the second argument or with specific dependencies.

- **On every render:** `useEffect(() => { ... })` (no dependency array).
- **Only when propA or stateB change:** `useEffect(() => { ... }, [propA, stateB])`.

36. How can you mimic componentWillUnmount using useEffect?

By returning a cleanup function from the useEffect hook. This function runs when the component unmounts or before the effect runs again.

```
useEffect(() => { const subscription = props.source.subscribe(); // Cleanup function return () => { subscription.unsubscribe(); }; }, [props.source]);
```

37. What is the "dependency array" in useEffect?

It's the second argument passed to useEffect. It's an array of values that the effect depends on. The effect will only re-run if any of these values have changed since the last render. An empty array [] means the effect runs only once after the initial mount.

38. What happens if you forget to specify the dependency array in useEffect?

The effect will run after every single render of the component (both mount and update). This can lead to performance issues or infinite loops if the effect itself triggers a state update.

39. Why do you sometimes get an infinite loop when using useEffect?

An infinite loop occurs when an effect updates a state variable that is also in its dependency array. The effect runs, updates the state, which causes a re-render, which causes the effect to run again because the dependency changed, and so on.

40. What is the purpose of the useLayoutEffect hook and how is it different from useEffect?

Both have the same signature, but they fire at different times.

- **useLayoutEffect** fires synchronously after all DOM mutations but before the browser has painted the screen. Use it to read layout from the DOM and synchronously re-render.
- **useEffect** fires asynchronously after the browser has painted the screen. Use it for most side effects (data fetching, subscriptions) to avoid blocking visual updates.

41. What was the purpose of the componentWillReceiveProps lifecycle method and why was it deprecated?

It was used to react to prop changes before a re-render. It was deprecated (and renamed to UNSAFE_componentWillReceiveProps) because it was often misused and led to bugs and inconsistencies. The recommended replacements are getDerivedStateFromProps (for rare cases) or componentDidUpdate.

42. What is the getDerivedStateFromProps lifecycle method used for?

It's a static method that is invoked right before calling the **render** method, both on the initial mount and on subsequent updates.

It should return an object to update the state, or **null** to update nothing. It exists for the rare case where the state depends on changes in props over time.

43. Why is getDerivedStateFromProps a static method?

Being static prevents you from accessing the component instance (**this**). This was intentional to steer developers away from side-effects and impure operations in this method, making it easier to reason about.

44. What is the getSnapshotBeforeUpdate lifecycle method used for?

It is invoked right before the most recently rendered output is committed to the DOM (e.g., before **componentDidUpdate**).

It enables your component to capture some information from the DOM (e.g., scroll position) before it is potentially changed. Any value returned by this method will be passed as a parameter to **componentDidUpdate**.

45. How do you handle errors in a class component?

By using the **componentDidCatch** lifecycle method or the static **getDerivedStateFromError** method.

These are called **Error Boundaries**. They are class components that catch JavaScript errors anywhere in their child component tree, log those errors, and display a fallback UI instead of the component tree that crashed.

46. What are the Rules of Hooks?

There are two main rules for using Hooks in React:

- **Only call Hooks at the top level:** Do not call Hooks inside loops, conditions, or nested functions. Always use them at the top level of your React function to ensure React can properly track their order between re-renders.
- **Only call Hooks from React functions:** You can only call Hooks from React functional components or custom Hooks. Do not call them from regular JavaScript functions.

These rules ensure that React maintains a consistent order of Hooks calls and can correctly preserve state across re-renders.

47. How does the useState hook work? What does it return?

The **useState** hook allows you to add state to a functional component.

```
const [state, setState] = useState(initialValue);
```

It returns an array with two elements:

- The current state value.
- A function to update that state value.

```
const [count, setCount] = useState(0);

function increment() {
```

```
  setCount(count + 1);
}
```

When **setCount** is called, React schedules a re-render, and the component updates with the new state value.

48. Can you call useState conditionally? Why?

No, you cannot call **useState** conditionally.

Hooks must be called in the same order on every render. If you call a Hook inside a condition, the order might change between renders, causing React to lose track of which state belongs to which Hook.

Incorrect:

```
if (show) {
  const [data, setData] = useState([]);
}
```

Correct:

```
const [data, setData] = useState([]);
if (show) {
  // Use the state here
}
```

49. How do you update state based on the previous state using useState?

To update state based on its previous value, pass a callback function to the state updater function. This callback receives the previous state as its argument.

```
const [count, setCount] = useState(0);

function increment() {
  setCount(prevCount => prevCount + 1);
}
```

This ensures the update is based on the most recent state, especially when multiple updates occur in quick succession.

50. What is the purpose of the useReducer hook and when would you prefer it over useState?

The **useReducer** hook is used to manage complex state logic in a component, especially when the next state depends on the previous one or when there are multiple related state

transitions.

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Example:

```
function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      return state;
  }
}

const [state, dispatch] = useReducer(reducer, { count: 0 });
```

You would prefer useReducer over useState when:

- The state logic is complex or involves multiple sub-values.
- The next state depends on the previous state.
- You want a more structured approach similar to Redux-style state management.